

task_bridge

Görev panonuz ve tek doğruluk kaynağınız kusursuz bir uyum içinde — her iki yönde.

Özet

task_bridge, Go içinde yer alan, genel, bağımsız ve çift yönlü bir görev/panel senkronizasyon motorudur. Bir projenin **işlenebilir öğeler** SQLite tek doğruluk kaynağını, izleyici belgeleri ve uzak bir panoyla (ilk hedef: ClickUp; Jira/Linear planlı) deterministik son düzenleme kazansın, önce kuru çalıştır, asla bozma ilkeleriyle senkronize eder.

Kısa açıklama

Projeden bağımsız bir Go alt modülü; SQLite işlenebilir öğeler tek doğruluk kaynağını ↔ izleyici belgeleri ↔ uzak bir panoyla (öncelikle ClickUp) çift yönlü senkronize eder. Deterministik son düzenleme kazansın, önce kuru çalıştır, HMAC doğrulamalı webhook'lar; tüm kimlik bilgileri ve ID'ler tüketici tarafından çalışma zamanında enjekte edilir.

Uzun açıklama

Her ekip, sonunda aynı işin iki farklı kaydını tutar: gerçek olan — kod, belgeler, dahili bir veritabanı — ve yöneticilerin izlediği, ClickUp gibi bir panel. İki taraf da dokunulduğu anda birbirinden uzaklaşır ve elle düzeltmek, tam da kimsenin güvenilir şekilde yapmadığı türden sıkıcı, hataya açık bir iştir. task_bridge, bu açığı kapatmak için üç temsil biçimini tek bir sistem olarak ele alır ve bunları senkronize halde tutar: bir projenin **işlenebilir öğeler** SQLite **tek doğruluk kaynağı**, **izleyici belgeleri** ve bir **uzak panel** — ilk desteklenen panel ClickUp olup, Jira ve Linear gelecekte eklenecekler arasında yer alıyor. Senkronizasyon deterministiktir (son düzenleme kazansın), önce kuru çalıştırma yapar ve tek bir tavizsiz vaat üzerine inşa edilmiştir: veri bozulmasına veya kaybına asla izin vermez, hiçbir tarafı sessizce eski kalmış bırakmaz. Dikkatsiz bir senkronizasyonun bir haftalık

çalışmayı silebileceği bir alanda, bu güvenlik yaklaşımı her şey demektir. Mimari olarak, diğer projeler tarafından tüketilen katı bir alt modüldür ve anayasanın bağımsızlık sözleşmesine (§11.4.28) uygun olarak tamamen projeden bağımsızdır: sıfır projeye özel değer içerir ve tüm kimlik bilgileri, panel/klasör ID'leri, öge anahtarı alanları ve veritabanı yolları, tüketici tarafından çalışma zamanında pkg/config.Config aracılığıyla enjekte edilir. Modül, katmanlı bir yapıdadır: CLI (reconcile/push/pull/resolve/status/conflicts/init) ve uzun süre çalışan bir arka plan hizmeti (webhook alıcısı + zamanlanmış senkronizasyon); MIT lisanslı raksul/go-clickup üzerine ince bir istemci sarmalayıcısı; canlı API sorgularıyla panel/klasör URL'lerini ID'lere dönüştüren bir çözümleyici (URL dilbilgisi tahmini yok); yerel işlenebilir öğeler ile uzak görev alanları arasında eşleştirici; açık çağırma sonuçları sunan son düzenleme kazansın senkronizasyon motoru; ve X-Signature HMAC-SHA256 doğrulayan bir webhook alıcısı. Olgunluk düzeyi konusunda dürüştür: Bu, P1 iskeletidir — yapı, arayüzler, giriş noktaları ve bağımsızlık sınırları yerinde olmakla birlikte, senkronizasyon mantığı ve canlı ClickUp çağrıları henüz uygulanmamıştır (her sahte işlev, sahte veri kuralına uygun olarak açık bir “uygulanmadı” hatası döndürür).

Neden geliştirdik

Ekipler, işin “gerçek” durumunu kod/belgelerde tutarken, yöneticiler ClickUp gibi bir panelde yaşar — ve bu ikisi sürekli birbirinden uzaklaşır. task_bridge, onları tek bir sistem haline getirir; deterministik ve güvenli bir şekilde senkronize eder, böylece hiçbir taraf eski veya yanlış kalmaz.

Neden Oyun Değiştirici

Çift yönlü pano senkronizasyonu genellikle tek seferlik, sabit entegrasyonlar olarak ele alınır ve her ekip bunu kötü bir şekilde yeniden inşa eder. task_bridge ise bunu, katı veri güvenliği garantileriyle donatılmış, yeniden kullanılabilir, kimlik bilgileri enjekte edilen bir kütüphane olarak yeniden tanımlıyor — öncelikle kuru çalıştırma, deterministik son düzenleme kazanır, HMAC doğrulamalı olaylar — böylece herhangi bir proje, iç yapılarına bağlı kırılğan bir bağlayıcı yazmak yerine, güvenilir pano entegrasyonunu yalnızca yapılandırma enjekte ederek benimseyebilir.

Yenilikçi Yönler

- Üç yönlü çift taraflı senkronizasyon: SQLite SSoT ↔ takipçi belgeleri ↔ uzak pano.

- Tamamen bağımsız yapı (§11.4.28): projeye özgü değer yok; hepsi çalışma zamanında enjekte edilir.
- Kırılgan URL gramer ayrıştırması yerine canlı API URL→ID çözümlemesi.
- Canlı olaylar için HMAC-SHA256 doğrulamalı webhook alımı.

Zorluklar ve Çözümler

- **Üç kaynaktaki veri güvenliği:** Deterministik son düzenleme kazanır, öncelikle kuru çalıştırma ve açık çakışma sonuçlarıyla çözüldü.
- **Bağılantısız yeniden kullanılabilirlik:** pkg/config enjeksiyon sınırı aracılığıyla çözüldü (projeye özgü hiçbir şey gönderilmez).
- **Güvenilir pano tanımlama:** URL'lerin canlı API sorgularıyla ID'lere çözümlenmesiyle çözüldü.
- **Şeffaf iskele:** Uygulanmamış stub'ların açık "uygulanmadı" hataları döndürmesiyle çözüldü (sahte veri yok).

Teknoloji Yığını (Neden ve Nasıl)

- **Go** — motor, CLI (cmd/task_bridge) ve arka plan işlemi (cmd/task_bridged).
- **SQLite** — işlenebilir öğelerin tek gerçek kaynağı.
- **raksul/go-clickup (MIT)** — ClickUp taşıma katmanı sarmalayıcısı.
- **HMAC-SHA256** — webhook imza doğrulaması.
- **cron + webhook'lar** — arka plan işlemi uzlaştırması + canlı olay alımı.
- **pkg/config** — çalışma zamanı kimlik bilgisi/ID enjeksiyon sınırı.

Durum şeffaflığı: Bu bir **P1 iskele**dir — senkronizasyon mantığı henüz uygulanmadı. Hazır ürün olarak sunmayın.